Programação concorrente

1º semestre de 2026

Thread

Tema #02

Professor Marcelo Eustáquio

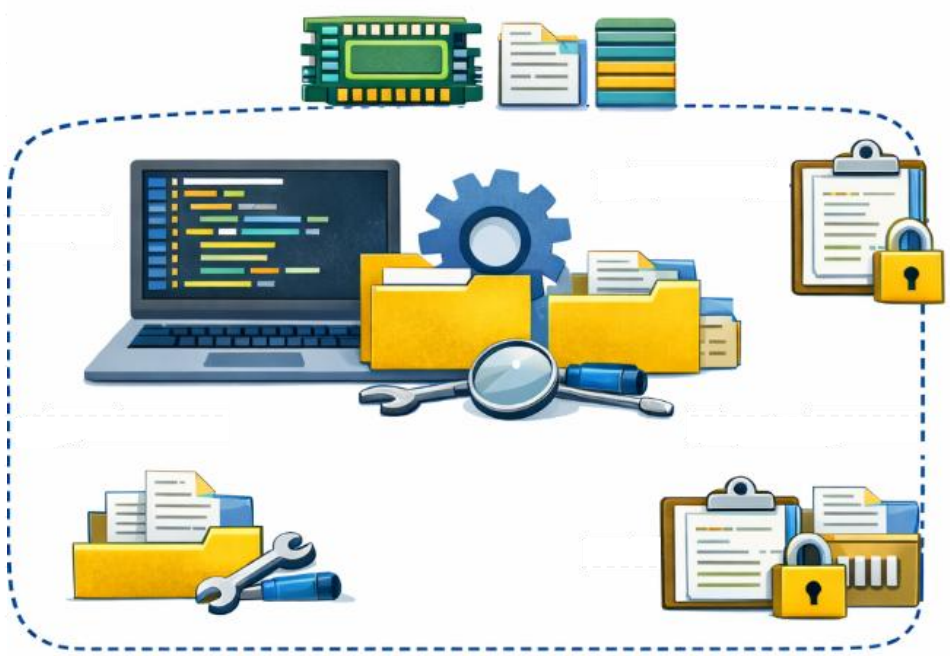
Thread

Um **processo** é um **programa em execução**. Quando um programa armazenado em disco (por exemplo, um arquivo .exe no Windows ou um binário no Linux) é carregado na memória e começa a ser executado pelo processador, ele passa a ser tratado pelo sistema operacional como um processo.

*Processos possuem **identificador único**, normalmente chamado de **PID (Process Identifier)**, que permite ao sistema operacional gerenciá-lo. O sistema controla diversos aspectos do processo, como seu estado de execução, a alocação de memória e o acesso aos dispositivos de entrada e saída.*

Thread

Um processo não é apenas o código do programa. Ele inclui também **todos os recursos necessários para sua execução**, como o espaço de memória, variáveis, pilha de execução, arquivos abertos, registradores do processador e informações de controle mantidas pelo sistema operacional. Em outras palavras, o processo representa o **contexto completo de execução de um programa**.



E **Threads**?

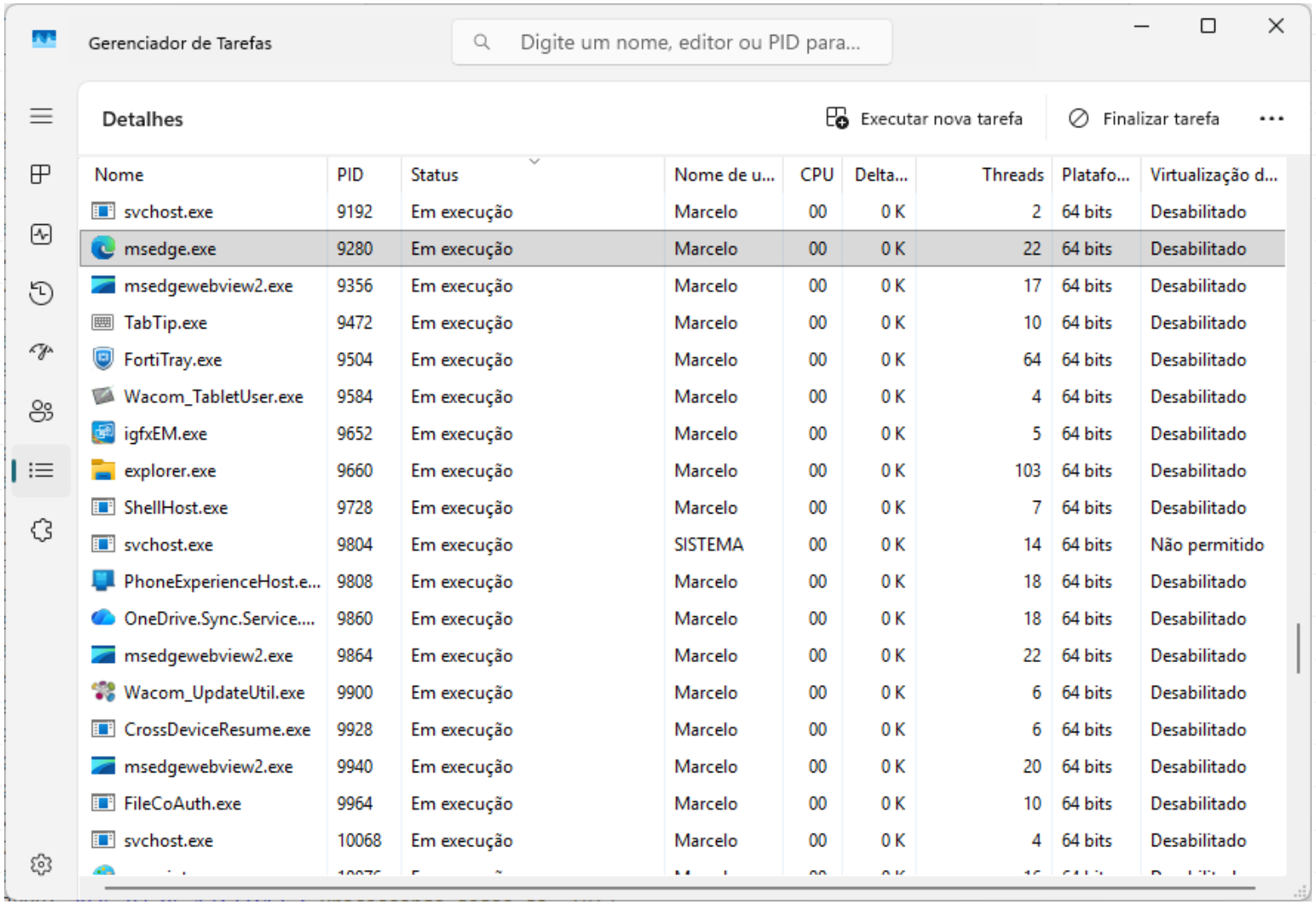
Threads são ***unidades de execução*** *dentro de um processo, ou seja, é um fluxo independente de instruções que ocorre dentro desse processo.*

Vamos detalhar...

Thread

As threads são utilizadas para permitir **concorrência** dentro de um mesmo programa. Enquanto processos diferentes possuem espaços de memória isolados, as threads de um mesmo processo ***compartilham o mesmo espaço de memória***, como variáveis globais, arquivos abertos e outros recursos. Cada thread, contudo, mantém seu próprio contador de programa, registradores e pilha de execução, o que permite que diferentes partes do código sejam executadas de forma independente.

Essas podem ser executadas de forma paralela.



Nome	PID	Status	Nome de u...	CPU	Delta...	Threads	Platafo...	Virtualização d...
svchost.exe	9192	Em execução	Marcelo	00	0 K	2	64 bits	Desabilitado
msedge.exe	9280	Em execução	Marcelo	00	0 K	22	64 bits	Desabilitado
msedgewebview2.exe	9356	Em execução	Marcelo	00	0 K	17	64 bits	Desabilitado
TabTip.exe	9472	Em execução	Marcelo	00	0 K	10	64 bits	Desabilitado
FortiTray.exe	9504	Em execução	Marcelo	00	0 K	64	64 bits	Desabilitado
Wacom_TabletUser.exe	9584	Em execução	Marcelo	00	0 K	4	64 bits	Desabilitado
igfxEM.exe	9652	Em execução	Marcelo	00	0 K	5	64 bits	Desabilitado
explorer.exe	9660	Em execução	Marcelo	00	0 K	103	64 bits	Desabilitado
ShellHost.exe	9728	Em execução	Marcelo	00	0 K	7	64 bits	Desabilitado
svchost.exe	9804	Em execução	SISTEMA	00	0 K	14	64 bits	Não permitido
PhoneExperienceHost.e...	9808	Em execução	Marcelo	00	0 K	18	64 bits	Desabilitado
OneDrive.Sync.Service....	9860	Em execução	Marcelo	00	0 K	18	64 bits	Desabilitado
msedgewebview2.exe	9864	Em execução	Marcelo	00	0 K	22	64 bits	Desabilitado
Wacom_UpdateUtil.exe	9900	Em execução	Marcelo	00	0 K	6	64 bits	Desabilitado
CrossDeviceResume.exe	9928	Em execução	Marcelo	00	0 K	6	64 bits	Desabilitado
msedgewebview2.exe	9940	Em execução	Marcelo	00	0 K	20	64 bits	Desabilitado
FileCoAuth.exe	9964	Em execução	Marcelo	00	0 K	10	64 bits	Desabilitado
svchost.exe	10068	Em execução	Marcelo	00	0 K	4	64 bits	Desabilitado

No **Windows** é possível visualizar as **threads de um processo**, mas isso não aparece diretamente na tela principal do Gerenciador de Tarefas. Siga esses passos:

1. Abra **Gerenciador de Tarefas** *(Ctrl + Shift + Esc).*

2. Vá na aba **Details (Detalhes)**.

3. Clique com o botão direito no cabeçalho das colunas.

4. Escolha **Select columns (Selecionar colunas)**.

5. Marque **Threads**.

Thread

A principal vantagem do uso de threads está na **eficiência e desempenho**. Criar uma thread costuma ser mais barato computacionalmente do que criar um processo completo, pois não é necessário duplicar todo o espaço de memória do programa. Além disso, em sistemas com múltiplos núcleos de processamento, ***diferentes threads podem ser executadas em paralelo***, aumentando o aproveitamento do hardware. Logo, aplicações que realizam múltiplas tarefas simultâneas (como servidores web, aplicações gráficas ou sistemas de processamento de dados) frequentemente utilizam múltiplas threads.

Threads são usadas para permitir que um programa execute várias tarefas simultaneamente, melhorando a eficiência e o desempenho.

Thread

O **número de threads** que podem ser executadas simultaneamente depende principalmente da quantidade de unidades de processamento disponíveis no hardware, ou seja, dos núcleos da CPU (cores) e do suporte a multithreading do processador.

1	(Núcleos do processador) Cada **núcleo físico da CPU** pode executar **uma thread por vez**. Portanto, se um processador possui: *4 núcleos*, ele pode executar *4 threads simultaneamente*.
2	(Multithreading do processador) Alguns processadores possuem tecnologia de **multithreading simultâneo (SMT).** Nesse caso, cada núcleo físico pode executar *mais de uma thread lógica ao mesmo tempo*.

Assim, o número máximo aproximado de threads que podem executar simultaneamente é:

$$Threads\ simultâneas = núcleos\ físicos \times threads\ por\ núcleo$$

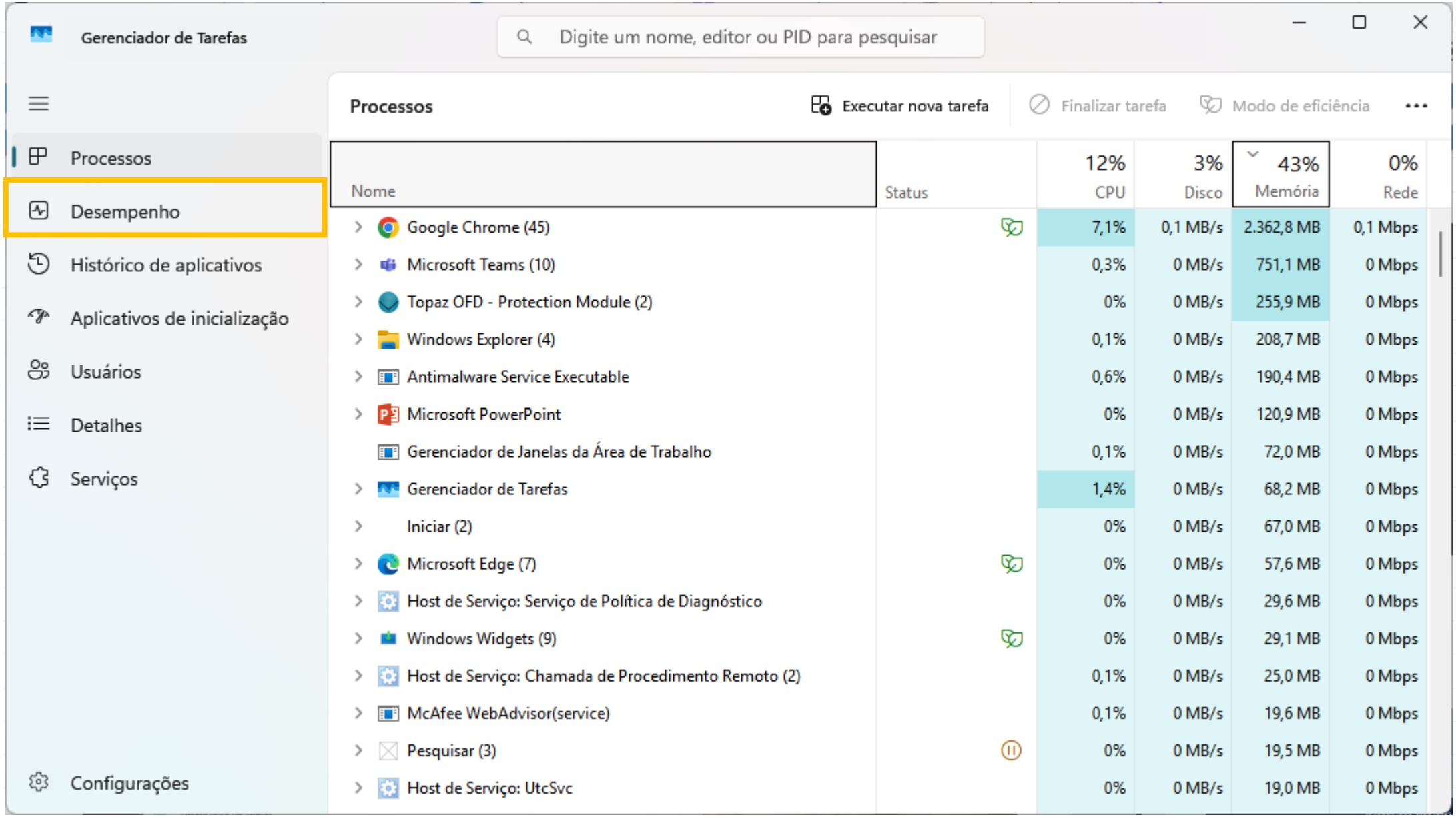
Essas informações podem ser acessadas pelo **Gerenciador de Tarefas**...

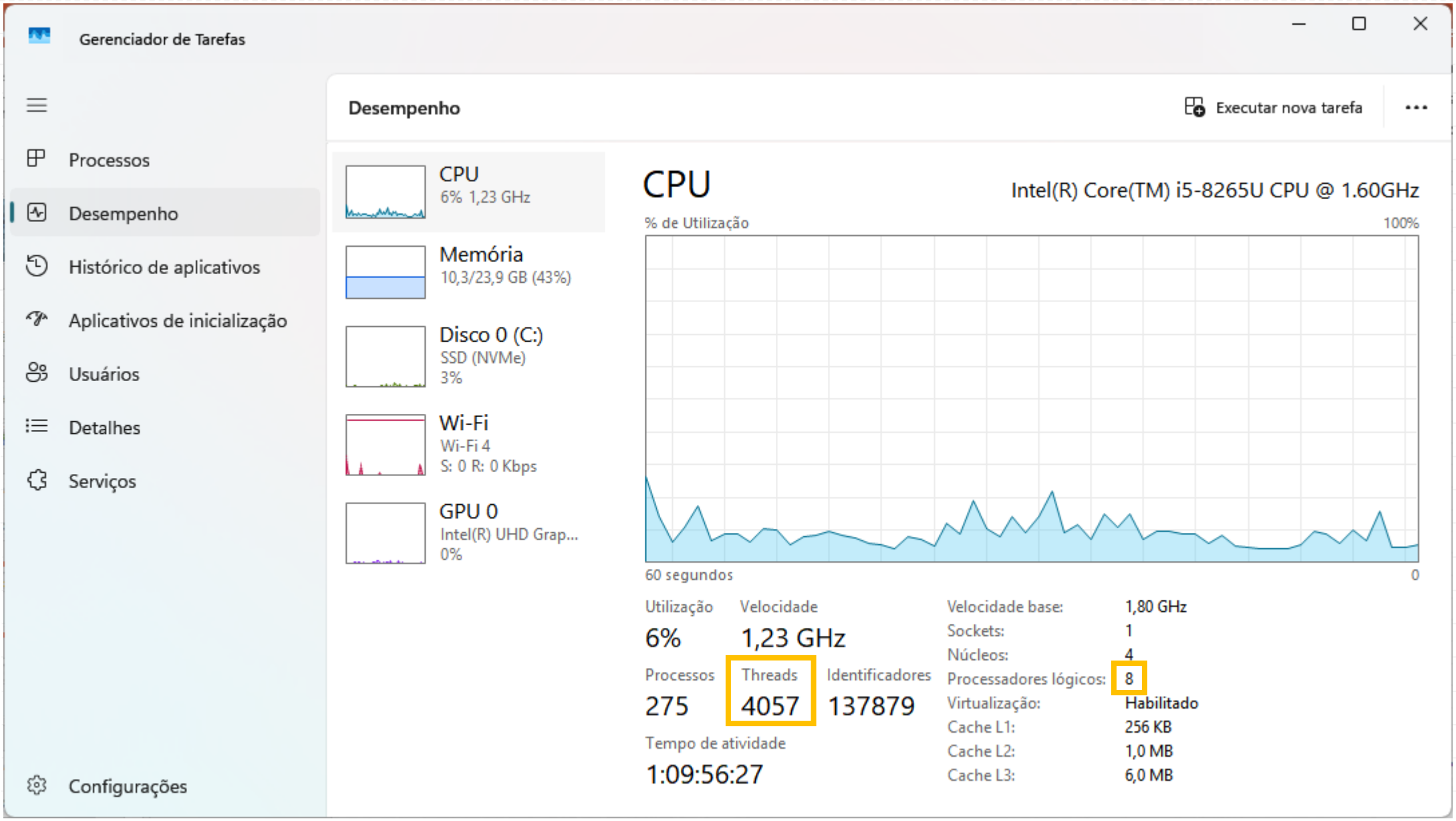
Thread

Os principais sistemas operacionais para desktops e servidores têm recursos nativos que permitem verificar o número de threads do processador em uso. No Windows, essa informação aparece no **Gerenciador de Tarefas** (no macOS e no Linux existem comandos próprios que revelam esse detalhe).

No Microsoft Windows

Digite "**Gerenciador de Tarefas**" no campo de busca da Barra de Tarefas ou utilize as teclas de atalho de teclado ***Ctrl + Shift + Esc***. A seguir, com o Gerenciador de Tarefas aberto, vá em "Desempenho" e "CPU". A quantidade de threads é informada abaixo do gráfico de desempenho, no campo "Processadores Lógicos".


Gerenciador de Tarefas
Digite um nome, editor ou PID para pesquisar
Processos
Desempenho
Histórico de aplicativos
Aplicativos de inicialização
Usuários
Detalhes
Serviços
Configurações
Processos
Executar nova tarefa
Finalizar tarefa
Modo de eficiência
Nome
Status
12% CPU
3% Disco
43% Memória
0% Rede
Google Chrome (45) 7,1% 0,1 MB/s 2.362,8 MB 0,1 Mbps
Microsoft Teams (10) 0,3% 0 MB/s 751,1 MB 0 Mbps
Topaz OFD - Protection Module (2) 0% 0 MB/s 255,9 MB 0 Mbps
Windows Explorer (4) 0,1% 0 MB/s 208,7 MB 0 Mbps
Antimalware Service Executable 0,6% 0 MB/s 190,4 MB 0 Mbps
Microsoft PowerPoint 0% 0 MB/s 120,9 MB 0 Mbps
Gerenciador de Janelas da Área de Trabalho 0,1% 0 MB/s 72,0 MB 0 Mbps
Gerenciador de Tarefas 1,4% 0 MB/s 68,2 MB 0 Mbps
Iniciar (2) 0% 0 MB/s 67,0 MB 0 Mbps
Microsoft Edge (7) 0% 0 MB/s 57,6 MB 0 Mbps
Host de Serviço: Serviço de Política de Diagnóstico 0% 0 MB/s 29,6 MB 0 Mbps
Windows Widgets (9) 0% 0 MB/s 29,1 MB 0 Mbps
Host de Serviço: Chamada de Procedimento Remoto (2) 0,1% 0 MB/s 25,0 MB 0 Mbps
McAfee WebAdvisor(service) 0,1% 0 MB/s 19,6 MB 0 Mbps
Pesquisar (3) 0% 0 MB/s 19,5 MB 0 Mbps
Host de Serviço: UtcSvc 0% 0 MB/s 19,0 MB 0 Mbps


Gerenciador de Tarefas
Processos
Desempenho
Histórico de aplicativos
Aplicativos de inicialização
Usuários
Detalhes
Serviços
Configurações
Desempenho
Executar nova tarefa
CPU
6% 1,23 GHz
Memória
10,3/23,9 GB (43%)
Disco 0 (C:)
SSD (NVMe)
3%
Wi-Fi
Wi-Fi 4
S: 0 R: 0 Kbps
GPU 0
Intel(R) UHD Grap...
0%
CPU
Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz
% de Utilização
100%
60 segundos
0
Utilização
6%
Velocidade
1,23 GHz
Processos
275
Threads
4057
Identificadores
137879
Tempo de atividade
1:09:56:27
Velocidade base: 1,80 GHz
Sockets: 1
Núcleos: 4
Processadores lógicos: 8
Virtualização: Habilitado
Cache L1: 256 KB
Cache L2: 1,0 MB
Cache L3: 6,0 MB

Thread

O número de processadores lógicos determina quantas threads podem ser executadas em paralelo, enquanto o sistema operacional gerencia a alternância de threads quando há mais threads do que processadores lógicos.

A biblioteca **"os"** do Python pode ser usada para obter o números de processadores (lógicos) ... :

```
import os

# Obtém o número de processadores lógicos
num_processadores_logicos = os.cpu_count()
print(f"Número de processadores lógicos: {num_processadores_logicos}")
```

Thread

Julgue a afirmação a seguir, classificando-a como *correta* (**C**) ou *errada* (**E**):

Em sistemas com múltiplos núcleos, a criação de threads resulta em aumento proporcional do desempenho da aplicação, uma vez que o sistema operacional é capaz de executar as threads simultaneamente sem necessidade de alternância de contexto.

Gabarito: Item ERRADO**.**

A criação de múltiplas threads não garante aumento **proporcional** de desempenho. Embora sistemas com múltiplos núcleos permitam a execução simultânea de várias threads, esse ganho depende de diversos fatores, como o **número de núcleos disponíveis**, a **natureza paralelizável da tarefa**, o **compartilhamento de recursos** e a **sobrecarga de sincronização**.

Além disso, quando o número de threads excede o número de **processadores lógicos**, o sistema operacional precisa realizar **alternância de contexto (context switching)** para compartilhar o tempo de CPU entre as threads. Mesmo quando há núcleos suficientes, ainda podem ocorrer trocas de contexto devido ao escalonamento do sistema operacional, interrupções ou bloqueios de recursos. Portanto, não é correto afirmar que a criação de threads resulta necessariamente em aumento proporcional de desempenho nem que sua execução ocorre sem alternância de contexto.

Em Python, o uso de **Threads** pode ser operacionalizado a partir do módulo **threading.**

Vamos detalhar...

Thread

Uma boa estratégia para utilizar de threads na linguagem Python consiste na implementação de funções, como a seguinte, que simula uma tarefa que demora (aproximadamente) 2 segundos para concluir:

```
import threading
import time

def tarefa():
    print("Início")
    time.sleep(2) # O tempo de espera não precisa ser valor inteiro (pode ser “float”)
    print("Fim")
```

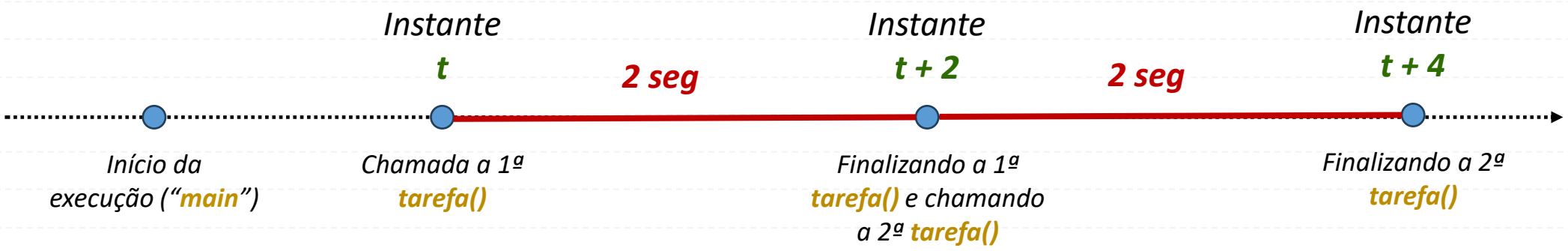
*O número 2 passado como argumento para sleep() indica a quantidade de segundos que a execução deve ser pausada. Ou seja, **time.sleep(2)** pausará a execução por exatos **2 segundos**. Observação: O tempo de pausa não é garantido com precisão absoluta, porque depende de escalonamento do sistema operacional, carga do processador e resolução do temporizador do sistema.*

```python
import threading
import time

def tarefa():
    print("Início")
    time.sleep(2) # O tempo de espera não precisa ser valor inteiro (pode ser “float”)
    print("Fim")
```

Linha do tempo, considerando que **tarefa()** foi evocada do bloco principal do código-fonte:

```python
import threading
import time

def tarefa():
    print("Início")
    time.sleep(2) # O tempo de espera não precisa ser valor inteiro (pode ser “float”)
    print("Fim")
```

Linha do tempo, considerando que **tarefa()** foi evocada **duas vezes** do bloco principal do código-fonte:



*Caso as tarefas sejam **independentes**, elas poderiam ser executadas de forma simultânea...*

Thread

Em python, a forma mais simples (e comum) para lidar com thread é usar a classe **Thread** (com “T” maiúsculo) do módulo **threading** e definir a função que será executada pela thread.

```
import threading
import time

def tarefa():
    print("Início")
    time.sleep(2) # O tempo de espera não precisa ser valor inteiro (pode ser “float”)
    print("Fim")

# Thread principal (linha do tempo principal)

thread = threading.Thread(target=tarefa) # Cria thread que executará a função `tarefa`
thread.start()  # Inicia a thread
thread.join() # Aguarda a conclusão da thread
print("Thread principal finalizada")
```

thread.py > ...

```python
import threading
import time

def tarefa():
    print("Início")
    time.sleep(2) # O tempo de espera não precisa ser valor inteiro (pode ser “float”)
    print("Fim")

# Thread principal (linha do tempo principal)

thread = threading.Thread(target=tarefa) # Cria thread que executará a função `tarefa`
thread.start()  # Inicia a thread
thread.join() # Aguarda a conclusão da thread
print("Thread principal finalizada")
```

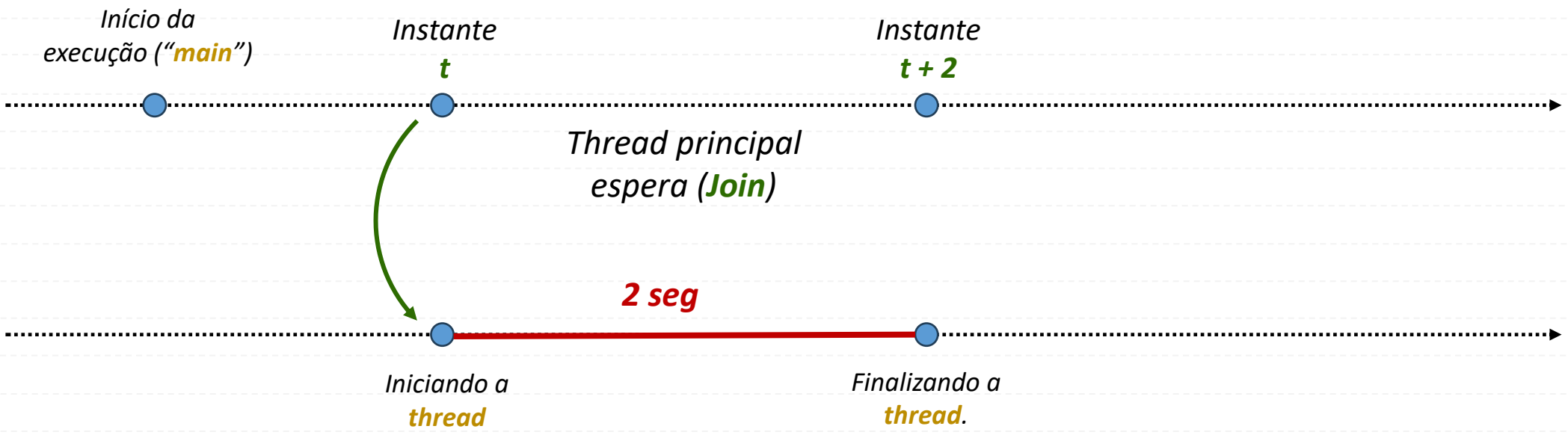
PROBLEMS DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
PS C:\Users\Admin\Desktop> python thread.py
Início
Fim
Thread principal finalizada
PS C:\Users\Admin\Desktop>
```

Thread

Durante a chamada à função, com a execução de thread.start(), existirão duas threads executando de forma simultânea, sendo a primeira a principal (linha do tempo principal) e a segunda uma ramificação da principal, que existirá por cerca de 2 segundos.



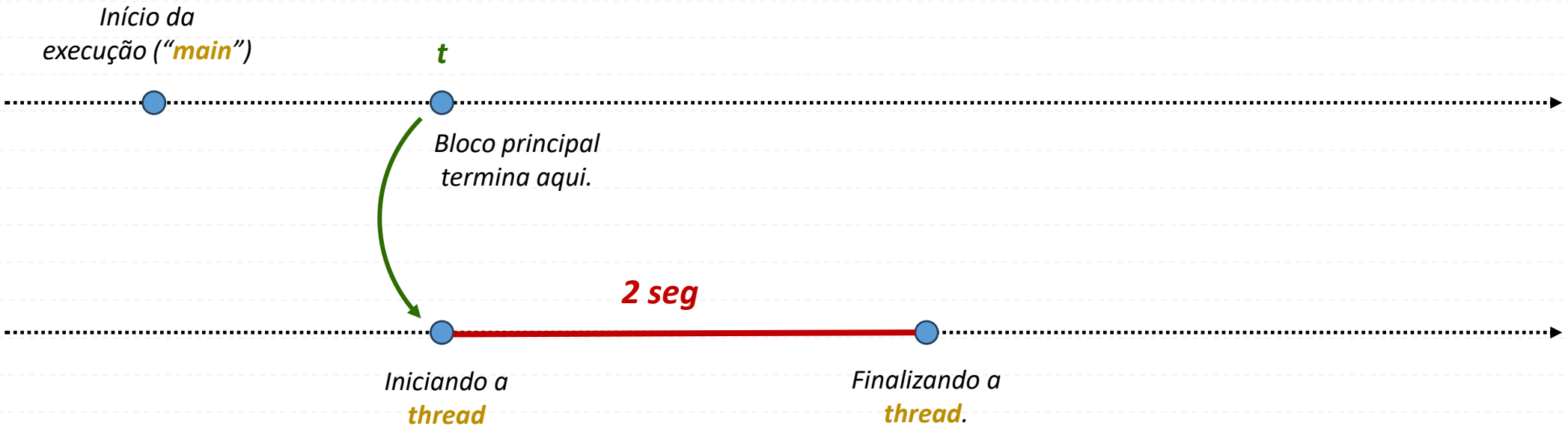
Thread

Explique o funcionamento do código a seguir, em que o método “Join” não foi evocado.

```
import threading
import time

def tarefa():
    print("Início")
    time.sleep(2) # O tempo de espera não precisa ser valor inteiro (pode ser “float”)
    print("Fim")

# Thread principal (linha do tempo principal)

thread = threading.Thread(target=tarefa)
thread.start()
print("Thread principal finalizada")
```

Thread

Explique o funcionamento do código (...), em que o método "**Join**" não foi evocado.



thread.py > ...

```python
import threading
import time

def tarefa():
    print("Início")
    time.sleep(2)
    print("Fim")

# Thread principal (linha do tempo principal)

thread = threading.Thread(target=tarefa)
thread.start()
#thread.join()
print("Thread principal finalizada")
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
PS C:\Users\Admin\Desktop> python thread.py
Início
Thread principal finalizada
Fim
PS C:\Users\Admin\Desktop>
```

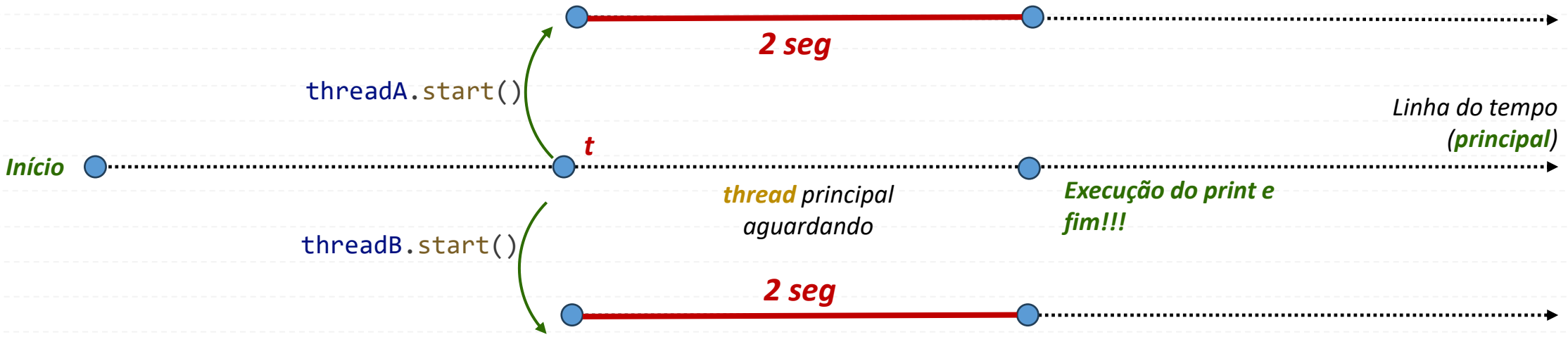
Thread

Considerando a mesma função **tarefa( )** utilizada nos exemplos anteriores, threads diferentes podem ser disparadas simultaneamente, como as que foram indicadas a seguir.

```
import threading
import time

def tarefa():
    print("Início")
    time.sleep(2) # O tempo de espera não precisa ser valor inteiro (pode ser “float”)
    print("Fim")

threadA = threading.Thread(target=tarefa)
threadB = threading.Thread(target=tarefa)
threadA.start()
threadB.start()
threadA.join()
threadB.join()
print("Thread principal finalizada")
```

Thread

Considerando a mesma função **tarefa( )** utilizada nos exemplos anteriores, threads diferentes podem ser disparadas simultaneamente, como as que foram indicadas a seguir. Nesse caso, o tempo total de execução será pouco (muito pouco) maior que 2 segundos.



Acima (e no código-fonte), a segunda entre as threads “threadA” e “threadB” é disparada sem que a primeira tenha sua execução finalizada.

Thread

Parâmetros podem ser passados para uma thread. Para tal, deve-se utilizar o atributo **args**, que recebe a lista de parâmetros da função

```
def saudacao(nome, tempo):
    print(f"Olá, {nome}")
    time.sleep(tempo)
    print(f"Tchau, {nome}")

# Cria e inicia a thread com argumentos

thread = threading.Thread(target = saudacao, args = ("Maria", 2))
thread.start()
thread.join()

print("Thread principal finalizada")
```

Thread

Como será o fluxo de mensagens (prints) do seguinte fragmento de código?

```
import threading
import time

def saudacao(nome, tempo):
    print(f"Olá, {nome}")
    time.sleep(tempo)
    print(f"Tchau, {nome}")

threadA = threading.Thread(target = saudacao, args = ("Ana", 5))
threadB = threading.Thread(target = saudacao, args = ("Beatriz", 2))
threadA.start()
threadB.start()
threadA.join()
threadB.join()

print("Thread principal finalizada")
```

No VSCode...

```
PS C:\Users\Admin\Desktop> python thread.py
Olá, Ana
Olá, Beatriz
Tchau, Beatriz
Tchau, Ana
Thread principal finalizada
PS C:\Users\Admin\Desktop>
```

O tempo total gasto seria de aproximadamente 5 segundos, uma vez que as threads seriam disparadas simultaneamente e executadas de forma paralela.

Thread

Modifique o código-fonte a seguir para que as threads sejam executadas de forma sequencial.

```
import threading
import time

def saudacao(nome, tempo):
    print(f"Olá, {nome}")
    time.sleep(tempo)
    print(f"Tchau, {nome}")

threadA = threading.Thread(target = saudacao, args = ("Ana", 5))
threadB = threading.Thread(target = saudacao, args = ("Beatriz", 2))
threadA.start()
threadB.start()
threadA.join()
threadB.join()

print("Thread principal finalizada")
```

Thread

Entretanto, o compartilhamento de memória também traz desafios. Quando duas ou mais threads acessam ou modificam os mesmos dados ao mesmo tempo, podem ocorrer **condições de corrida (*race conditions*)**, levando a resultados incorretos ou inconsistentes. Para evitar esse tipo de problema, utilizam-se mecanismos de **sincronização**, como *mutex*, *semaphores*, *locks* e *monitores*, que controlam o acesso concorrente aos recursos compartilhados.

Sincronização é o conjunto de mecanismos utilizados para coordenar a execução de múltiplas threads ou processos que compartilham recursos. Seu objetivo é garantir que o acesso a dados compartilhados ocorra de forma controlada, evitando inconsistências, condições de corrida (*race conditions*) e comportamentos inesperados.

Thread

Em Python, uma **variável global** é uma variável definida *fora de qualquer função ou bloco local*, tornando-se acessível a todas as funções e partes do programa dentro do mesmo módulo. Por estar declarada no escopo global, seu valor pode ser lido por qualquer função, desde que esteja visível naquele contexto. Por padrão, funções podem ler variáveis globais, mas não podem modificá-las diretamente sem declarar explicitamente que estão utilizando a variável global. Para isso, utiliza-se a palavra-chave **global**, que informa ao interpretador que a variável referenciada dentro da função pertence ao escopo global.

```
contador = 0   # variável global

def incrementar():
    global contador
    contador += 1

incrementar()
print(contador)
```

```python
import threading
import time

contador = 0

def incrementar():
    global contador
    for _ in range(10000):
        valor = contador        # 1. lê
        time.sleep(0.0000001)   # força troca de contexto
        valor = valor + 1       # 2. calcula
        contador = valor        # 3. grava

threads = []

for i in range(4):
    t = threading.Thread(target=incrementar)
    threads.append(t)
    t.start()

for t in threads: t.join()
print("Valor final:", contador)
```

TERMINAL

```
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10003
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10002
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10002
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10002
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10001
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10001
PS C:\Users\Admin\Desktop>
```

Thread

Variáveis globais podem ser usadas de forma compartilhada por diferentes threads e, para tal, na função que será paralelizada, deve-se usar **global** para fazer referência ao espaço de memória compartilhado. Essa estrutura será detalhada a seguir...

```
import threading

# Variável global
contador = 0
# Cria um lock

lock = threading.Lock()
```

lock = threading.Lock()
cria um objeto de lock que será usado para sincronizar o acesso à variável.

```python
import threading
import time

contador = 0
lock = threading.Lock()

def incrementar():
    global contador
    for _ in range(10000):
        with lock:
            valor = contador          # 1. lê
            time.sleep(0.0000001)     # força troca de contexto
            valor = valor + 1         # 2. calcula
            contador = valor          # 3. grava

threads = []
for i in range(4):
    t = threading.Thread(target=incrementar)
    threads.append(t)
    t.start()

for t in threads:t.join()
print("Valor final:", contador)
```

TERMINAL

```
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10003
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10002
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10002
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10002
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10001
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 10001
PS C:\Users\Admin\Desktop> python thread.py
Valor final: 40000
PS C:\Users\Admin\Desktop>
```

```python
import threading
import time

contador = 0
lock = threading.Lock()

def incrementar():
    global contador
    for _ in range(10000):
        lock.acquire()              # adquire o lock
        valor = contador            # 1. lê
        time.sleep(0.0000001)       # força troca de contexto
        valor = valor + 1           # 2. calcula
        contador = valor            # 3. grava
        lock.release()              # libera o lock

threads = []
for i in range(4):
    t = threading.Thread(target=incrementar)
    threads.append(t)
    t.start()
for t in threads: t.join()
print("Valor final:", contador)
```

Thread

As threads podem ser classificadas em dois tipos principais: **threads de usuário** e **threads de núcleo (kernel)**. As threads de usuário são gerenciadas por bibliotecas no espaço do programa, sem intervenção direta do sistema operacional. Já as threads de núcleo são gerenciadas pelo próprio sistema operacional, que controla seu escalonamento e execução nos processadores. Muitos sistemas modernos utilizam modelos híbridos que combinam essas duas abordagens para equilibrar desempenho e controle.

Programação concorrente

1º semestre de 2026

Thread

Tema #02

Professor Marcelo Eustáquio

Q01 Considere o programa:

```
import threading
import time

x = 0

def tarefa():
    global x
    for _ in range(3):
        temp = x
        time.sleep(0.1)
        x = temp + 1
```

```python
threads = []

for _ in range(2):
    t = threading.Thread(target=tarefa)
    threads.append(t)
    t.start()

for t in threads: t.join()
print(x)
```

Responda:

a) Qual é o valor máximo possível de x ao final da execução?
b) Qual é o valor mínimo possível de x?
c) Explique detalhadamente um interleaving de execução que produza o valor mínimo.

Q02

Considere o programa:

```
saldo = 1000

def sacar(valor):
    global saldo
    if saldo >= valor:
        temp = saldo
        temp -= valor
        saldo = temp
```

Suponha duas threads executando sacar(500) simultaneamente.

A) Identifique a seção crítica.
B) Explique como pode ocorrer uma inconsistência no saldo.
C) Reescreva o código usando Lock.

Q03

Considere o programa:

```
import threading
import time

def tarefa1(): time.sleep(4)
def tarefa2(): time.sleep(3)
def tarefa3(): time.sleep(2)

t1 = threading.Thread(target=tarefa1)
t2 = threading.Thread(target=tarefa2)
t3 = threading.Thread(target=tarefa3)

inicio = time.perf_counter()

t1.start()
t2.start()
t3.start()
t1.join()
t2.join()
t3.join()
fim = time.perf_counter()
print(fim - inicio)
```

Considerando o código apresentado, determine o tempo mínimo teórico de execução, as condições em que esse tempo pode ser maior e como o número de núcleos da CPU influencia esse resultado.

Q04

Considere o programa:

```
import threading

def tarefa(nome):
    for i in range(2):
        print(nome, i)

t1 = threading.Thread(target=tarefa, args=("A",))
t2 = threading.Thread(target=tarefa, args=("B",))

t1.start()
t2.start()

t1.join()
t2.join()
```

Liste 2 sequências possíveis de saída e explique por que a ordem das mensagens não é determinística.

Q05 Explique o funcionamento do código-fonte a seguir:

```python
import threading

lockA = threading.Lock()
lockB = threading.Lock()

def tarefa1():
    lockA.acquire()
    lockB.acquire()
    print("Tarefa1 executando")
    lockB.release()
    lockA.release()

def tarefa2():
    lockB.acquire()
    lockA.acquire()
    print("Tarefa2 executando")
    lockA.release()
    lockB.release()
```

Q06

Suponha um programa que executa uma tarefa paralelizável em 8 threads, sendo que a máquina possui 4 núcleos físicos e 8 processadores lógicos.

A) Qual o número máximo de threads que podem executar simultaneamente?
B) O que acontece com as demais threads?
C) Explique o papel do escalonador do sistema operacional nesse cenário.

Q07

Julgue o item:

Em um sistema com múltiplos núcleos, o uso de locks para proteger seções críticas sempre melhora o desempenho de aplicações concorrentes, pois evita inconsistências causadas pelo acesso simultâneo à memória compartilhada.

Q08

Divida a multiplicação de duas matrizes 4 x 4 entre 4 threads.

Observação:

Suponha que as primeiras linhas do seu script sejam:

```
import threading

matriz_a = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]]
matriz_b = [[16, 15, 14, 13], [12, 11, 10, 9], [8, 7, 6, 5], [4, 3, 2, 1]]
resultado = [[0]*4 for _ in range(4)]
lock = threading.Lock()
```

Q09

Determine a lista com todos os números primos entre 1 e 1.000.000 usando 5 threads.

Observação:

Divida o intervalo entre as threads e combine os resultados. Ademais, considere que as primeiras linhas do seu script sejam:

```
import threading
import math

primos = []
lock = threading.Lock()
```

Q10

Escreva script em Python para determinar a média e o desvio padrão de um conjunto de 100.000 números gerados aleatoriamente usando 3 threads. Gere uma planilha do Excel para confirmar o resultado que você obteve.

Q11

Escreva um script em Python para resolver um sistema de $\boldsymbol{n}$ equações lineares e $\boldsymbol{n}$ incógnitas usando o **Método de Cramer**, distribuindo o processamento em $\boldsymbol{n}$ threads.

Observação:

Dado um sistema linear na forma matricial $AX = B$ onde A é a matriz dos coeficientes $(n \times n)$, X é o vetor das incógnitas $(n \times 1)$, B é o vetor dos termos independentes $(n \times 1)$. O método de Cramer calcula cada incógnita x_i como

$$x_i = \frac{\det(A_i)}{\det(A)}$$

Onde det(A) é o determinante de A, e A_i é a matriz obtida substituindo a coluna i de A pelo vetor B.

```python
import random
import threading
import pandas as pd
import math

N = 100000  # quantidade de números
dados = [random.random() for _ in range(N)] # gerar números aleatórios
NUM_THREADS = 3 # número de threads

# dividir dados

partes = []
tamanho = N // NUM_THREADS

for i in range(NUM_THREADS):
    inicio = i * tamanho
    if i == NUM_THREADS - 1:
        fim = N
    else:
        fim = (i + 1) * tamanho
    partes.append(dados[inicio:fim])
```

```python
# resultados parciais

somas = [0] * NUM_THREADS

somas_quadrados = [0] * NUM_THREADS

def calcular(parte, idx):
    soma = sum(parte)
    soma_quad = sum(x*x for x in parte)
    somas[idx] = soma
    somas_quadrados[idx] = soma_quad

threads = []

# criar threads

for i in range(NUM_THREADS):
    t = threading.Thread(target=calcular, args=(partes[i], i))
    threads.append(t)
    t.start()
```

```python
# aguardar threads

for t in threads: t.join()

# combinar resultados

soma_total = sum(somas)
soma_quad_total = sum(somas_quadrados)
media = soma_total / N
variancia = (soma_quad_total / N) - (media ** 2)
desvio_padrao = math.sqrt(variancia)

print("Média:", media)
print("Desvio padrão:", desvio_padrao)

# salvar no Excel para conferência

df = pd.DataFrame({"numeros": dados})
df.loc[0, "media_calculada"] = media
df.loc[0, "desvio_padrao_calculado"] = desvio_padrao
df.to_excel("verificacao_threads.xlsx", index=False)
print("Planilha 'verificacao_threads.xlsx' gerada.")
```